

UPV/EHU

ESCUELA DE INGENIERÍA DE BILBAO



SISTEMAS DE GESTIÓN DE SEGURIDAD DE SISTEMAS DE INFORMACIÓN

ENTREGA 2

Profesor:

Mikel Egaña Aranguren

Alumnos:

Carolina Estefanía Jiménez Godoy

Gonzalo Pérez Gómez

Iratxe Urrutia Sordo

Lucía Gutiérrez Diez

Proyecto auditado:

<https://github.com/GonzaloPerezGomez/SGSSI>

24 de noviembre de 2024

Índice

1. Introducción	2
2. Primera auditoría	2
3. Vulnerabilidades	3
3.1. Rotura de control de acceso	3
3.1.1. Control de acceso por roles	3
3.1.2. Denegación de acceso mediante URL a js y css	3
3.2. Fallos criptográficos	4
3.2.1. Almacenamiento de Contraseñas	4
3.2.2. Contraseña HTML	5
3.2.3. Encriptado de datos sensibles	5
3.2.4. Algoritmos débiles u obsoletos	6
3.3. Inyección	6
3.4. Diseño inseguro	7
3.4.1. “hard-coded credentials”(credenciales en el código fuente)	7
3.4.2. Protección insuficiente de las cookies	7
3.4.3. Reutilización de código seguro	8
3.4.4. Comentarios difamatorios en los javaScript	8
3.5. Configuración de seguridad insuficiente	8
3.5.1. Divulgación de información sensible sobre el servidor web	8
3.5.2. Encabezado X-Content-Type	8
3.5.3. Protección insuficiente contra framing	9
3.5.4. Gestión de errores para no desvelar información	9
3.5.5. Cabecera CSP	10
3.5.6. Pasar información por la URL(dirección web)	10
3.5.7. Cabecera Cache-Control	10
3.6. Componentes vulnerables y obsoletos	11
3.6.1. Actualización de versión de PHP	11
3.6.2. Actualización de versión de phpmyadmin	11
3.7. Fallos de identificación y autenticación	11
3.7.1. Contraseñas débiles o por defecto	11
3.7.2. Invalidación de sesiones después de logout o inactividad	12
3.7.3. Limitar intentos fallidos de login	13
3.7.4. Logear intentos fallidos de login	14
3.7.5. Uso de ID aleatorio para las sesiones	14
3.7.6. Protección contra ataques Cross-Site Request Forgery (CSRF)	14
3.7.7. Uso de reCAPTCHA	15
3.8. Fallos en la integridad de datos y software	17
3.8.1. Librerías no confiables	17
3.8.2. Copias de Seguridad	17
3.9. Fallos en la monitorización de la seguridad	17
4. Segunda auditoría	17
5. Conclusiones	18
6. Observaciones	18
7. Bibliografía	18

1. Introducción

Uno de los temas con mayor polémica estos últimos años, ha sido la seguridad de los datos personales en el mundo informático. Cada vez se le da más importancia a tener una buena seguridad en los sistemas web, para evitar vulnerabilidades que puedan derivar en ataques u obtención de información sensible. Mediante este segundo trabajo a entregar, se quiere identificar y solucionar las vulnerabilidades web desarrolladas en la entrega anterior que permitirán evitar posibles ataques en el futuro. Para esto, haremos uso de una herramienta de pen-testing ZAP que analiza sitios web en búsqueda de vulnerabilidades exponiendo un listado de vulnerabilidades descritas del informe OWASP 2021 que se encuentran en nuestro proyecto.

Para cumplir el objetivo del trabajo, hemos realizado dos auditorías. La primera sobre la web de la anterior entrega y la segunda después de solucionar las vulnerabilidades detectadas en la primera auditoría. La URL de Github del proyecto auditado es <https://github.com/GonzaloPerezGomez/SGSSI>.

Después de la detección de las debilidades y vulnerabilidades del proyecto web. Se realizará una búsqueda del tipo de problema y analizaremos sus causas. Se explicará en el informe a continuación los cambios realizados en la implementación para solucionar las vulnerabilidades mencionadas. Se explicarán también ciertos aspectos de seguridad que nuestro proyecto cumplía antes de la primera auditoría y otros aspectos de seguridad que no son captados por el programa ZAP.

Finalmente, comparamos los resultados de las dos auditorías, analizando las mejoras obtenidas y sacando conclusiones sobre la importancia de solucionar las vulnerabilidades de los sistemas web.

2. Primera auditoría

El objetivo de esta primera auditoría es detectar y registrar los fallos de seguridad y vulnerabilidades de nuestro sistema web. Para este apartado hemos hecho uso de la herramienta ZAP. ZAP es un proyecto de OWASP que se destaca por la gran variedad de acciones que tiene a su disposición que abarcan desde el escaneo automático hasta la intersección de peticiones y respuestas. Al ejecutar ZAP en nuestro proyecto web vemos que detecta las siguientes vulnerabilidades.

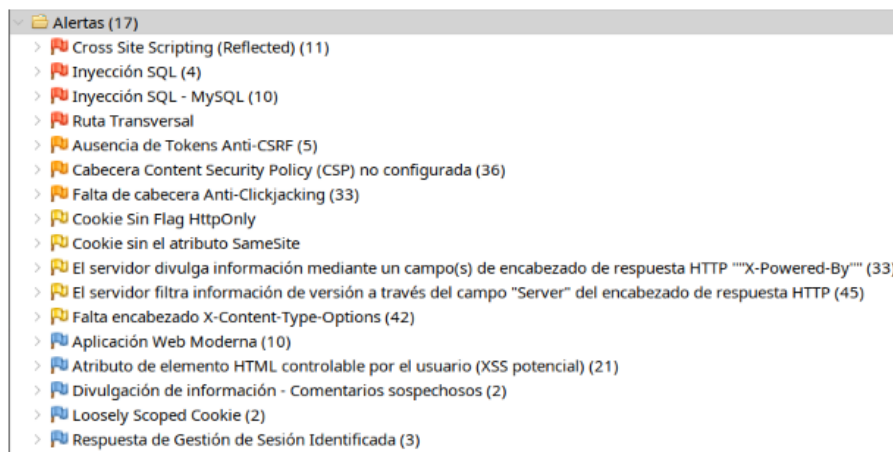


Figura 2.0.0.1: Primera auditoría - ZAP

Como se puede ver una de las brechas de seguridad más importantes de nuestra aplicación son las Inyecciones SQL. Por otra parte también se aprecia que la mayoría de vulnerabilidades se originan en la mala o nula configuración de las páginas web (cabeceras y tokens).

3. Vulnerabilidades

3.1. Rotura de control de acceso

3.1.1. Control de acceso por roles

En el proyecto, podemos encontrarnos con violaciones del principio de denegación por defecto. Este principio indica que el acceso solo tiene que estar disponible para ciertas personas, roles o capacidades. Sin embargo, en el proyecto no se ha tenido en cuenta este principio y todas las páginas pueden ser accedidas independientemente del tipo de usuario.

Para solucionar el problema, se ha implementado el acceso por roles. Para cada usuario, se guarda el rol que tiene (administrador, usuario) en la base de datos. De esa manera se antes de cargar la página comprobamos si el cliente es un administrador. Si no lo es, se le redirige a una página en la que pueda estar.

```
//comprueba si se ha iniciado sesion
if (!isset($_SESSION['randomID']) || $_SESSION['tipo'] != 'admin') {
    echo "<script> window.location.href = 'items.php';</script>";
    exit();
}
```

Figura 3.1.1.1: Comprobación de rol

```
CREATE TABLE `usuarios` (
  `idUsuario` int(11) NOT NULL,
  `nombre` text NOT NULL,
  `apellido` text NOT NULL,
  `numeroDNI` text NOT NULL,
  `letraDNI` text NOT NULL,
  `telefono` text NOT NULL,
  `nacimiento` text NOT NULL,
  `email` text NOT NULL,
  `usuario` text NOT NULL,
  `contrasena` text NOT NULL,
  `salt` text NOT NULL,
  `tipo` text DEFAULT 'user'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figura 3.1.1.2: Tabla usuarios

3.1.2. Denegación de acceso mediante URL a js y css

Es importante asegurarse de que algunos archivos como .js o .css no sean accesibles desde la URL para que ningún usuario pueda ver el código interno del sistema web. En la anterior entrega se podía acceder a estos archivos escribiendo `http://localhost:81/estilo.css` o `http://localhost:81/comprobacionDeDatos.js`.

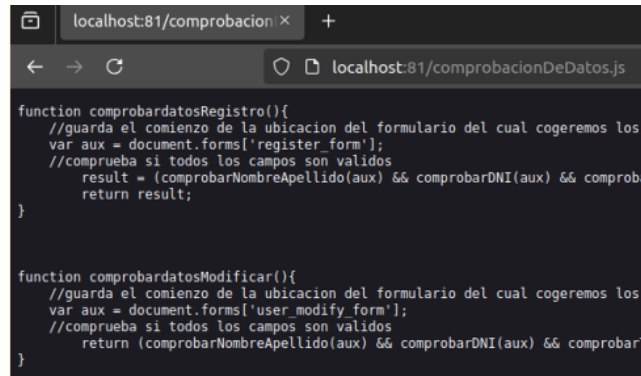


Figura 3.1.2.1: localhost:81/comprobacionDeDatos.js antes

Para solucionarlo, se ha denegado el acceso a ellos directamente desde la URL, permitiendo que el sistema web continúe usándolos.

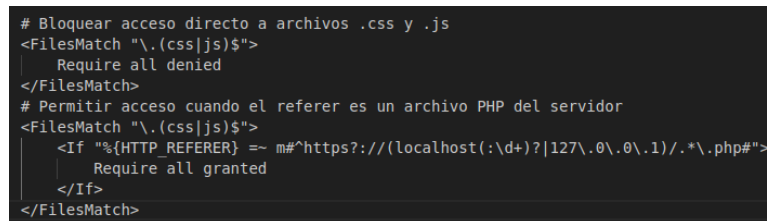
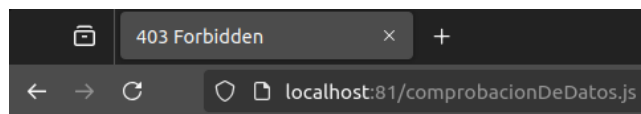


Figura 3.1.2.2: Bloqueo js y css



Forbidden

You don't have permission to access this resource.

Figura 3.1.2.3: localhost:81/comprobacionDeDatos.js ahora

3.2. Fallos criptográficos

Los fallos criptográficos son brechas de seguridad que pueden ser producto de varias circunstancias, como fallos en la arquitectura del sistema, mal uso de los algoritmos criptográficos, o incluso recurrir a algoritmos obsoletos o no seguros.

3.2.1. Almacenamiento de Contraseñas

En el caso de las contraseñas de los usuarios guardabamos las información sin encriptar, lo que puede causar que cualquiera que acceda a la base de datos se haga con las contraseñas de todos los usuarios.

usuario	contrasena
admin	1234

Figura 3.2.1.1: Almacenamiento antiguo de contraseñas

Para solucionarlo hemos cambiado el sistema de registro. En vez de guardar la contraseña en texto plano lo que se hace es primero generar una semilla aleatoria de 255 bytes. Está semilla se concatena al final de la

contraseña introducida por el usuario y se genera un hash de la cadena de caracteres resultante. El algoritmo de resumen usado es SHA256 ya que se considera un algoritmo suficientemente seguro.

En la base de datos se guarda el hash resultante y la semilla generada para poder comprarlos a futuro. Cuando un usuario quiere iniciar sesión se recupera el hash y la semilla. Se genera un hash con la semilla y la contraseña introducida y se compara con el hash almacenado. Si los hashes coinciden es que la contraseña es correcta, en caso contrario se deniega la sesión.

```
//generamos una semilla de 255 bytes
$salt = bin2hex(random_bytes(255));
//generamos el hash apartir de la contraseña mas la semilla
$contraseña_completa = $contraseña . $salt;
$hash_contraseña = hash("sha256", $contraseña_completa);
```

Figura 3.2.1.2: Hash de las contraseñas

contrasena	salt
3442ca9c2c37ae246209fbb7eb08dea4119298318b2633a6ff...	1fc8b42e1448273f2a5de0fb2dc24c4584354f32f55e55e622...

Figura 3.2.1.3: Almacenamiento actual de contraseñas

3.2.2. Contraseña HTML

Para finalizar con la protección de las contraseñas, veremos el uso del tipo “password” en el html. En nuestro proyecto, a la hora de introducir la contraseña en el formulario, se utilizaba un texto de tipo “text”. Esto suponía un problema, no solo porque la contraseña quedaba expuesta a simple vista para cualquier persona alrededor del usuario, sino por la baja seguridad con la que el navegador interactúa con la contraseña.

Para mejorar este aspecto, hemos cambiado el tipo de texto a un tipo “password”. De esa manera, al introducir la contraseña, el valor queda oculto por símbolos (*), haciéndola ilegible por otras personas. En cuanto al trato del valor de la contraseña por parte de los navegadores, comienzan a proteger y gestionar de manera más segura.

```
Contraseña:<br>
<input type="password" name="contrasena" id="contrasena" class="form-control form-control-lg"
value="" autocomplete="current-password" required> <br>
```

Figura 3.2.2.1: Implementacion de contraseña

3.2.3. Encriptado de datos sensibles

La encriptación de los datos sensibles tanto en la transmisión como en la base de datos es esencial. Antes almacenábamos toda la información de los usuarios en texto plano.

numeroDNI	letraDNI	telefono	nacimiento	email
12345678	z	123456789	1800-01-01	admin@gmail.com

Figura 3.2.3.1: Almacenamiento antiguo de datos sensibles

Para solucionar este problema hemos implementado las funciones openssl_encrypt() y openssl_decrypt() para encriptar y desencriptar los datos respectivamente. Ambas funciones requieren de un algoritmo, una clave y un vector de inicialización. En cuanto al algoritmo, en ambos casos se usa aes-256-cbc ya que se considera que es muy seguro. Tanto la clave como el vector de inicialización se declaran como variables de entorno de docker para evitar copiarlos en varias partes del código.

numeroDNI	letraDNI	telefono	nacimiento	email
tzVvh9ZzZK431yxqe/aEtA==	/jeahZ6NBZ3r+b0sAlNyrw==	oBV5rYlgpp6Ld96jt3K+XQ==	kjecf34FuTT0E7Ildump7Q==	3MPUsYIFc4y7a/VMAvxKPg==

Figura 3.2.3.2: Almacenamiento actual de datos sensibles

3.2.4. Algoritmos débiles u obsoletos

Cerciorarse de que los algoritmos de cifrado son seguros es muy importante. La utilización de algoritmos débiles u obsoletos produce que el sistema sea especialmente vulnerable ante ataques de fuerza bruta o ataques de colisión en el caso de los algoritmos de resumen. En nuestro caso, nos hemos asegurado que todos los algoritmos utilizados no tengan vulnerabilidades graves y que en la práctica sean irrompibles.

3.3. Inyección

En cuanto a la interacción con la base de datos no se hacía ningún tipo de saneamiento de la información introducida por el usuario en los formularios, por lo que el sistema era muy vulnerable a ataques de inyección SQL. Para evitar esta vulnerabilidad se ha introducido la utilización de las funciones `prepare()` y `bind_param()` en todas las consultas SQL para evitar cualquier tipo de input maliciosos.

```
//guarda la instrucción de SQL que quiere utilizar, en este caso un select
$sql = "SELECT usuario from usuarios where usuario = '" . $usuario . "'";
```

Figura 3.3.0.1: Ejemplo de consulta sin sanear

```
//guarda la instrucción de SQL que quiere utilizar, en este caso un select
$sql = "SELECT usuario from usuarios where usuario = ? OR numeroDNI = ?";
$stmt = $conn->prepare($sql);
$stmt->bind_param('ss', $usuario, $DNI);
```

Figura 3.3.0.2: Ejemplo de consulta saneada

Un ejemplo de inyección que funcionaba antes de sanear la información de las consultas:



Figura 3.3.0.3: Ejemplo de inyección

Esta inyección refleja claramente porque es necesario protegerse contra las inyecciones. En este caso permite que cualquier atacante inicie sesión en el sistema sin necesidad de una cuenta.

En conjunto a la parametrización de las consultas de la base de datos se ha añadido la función de `htmlspecialchars()` que sanitiza la información introducida por el usuario con el fin de evitar los ataques de inyección de código HTML y JavaScript en contextos como la visualización del contenido en la página web. El método `htmlspecialchars()` convierte los caracteres especiales en sus correspondientes caracteres HTML.

```

// guardar la información del formulario
//guarda el título del libro
$title = encrypt(plaintext: htmlspecialchars(string: $_POST['titulo']));
//guarda el autor del libro
$autor= encrypt(plaintext: htmlspecialchars(string: $_POST['autor']));
//guarda la fecha de publicación del libro
$f_publicacion = encrypt(plaintext: htmlspecialchars(string: $_POST['f_publicacion']));
//guarda el ISBN del libro
$ISBN= encrypt(plaintext: htmlspecialchars(string: $_POST['ISBN']));
//guarda el número de páginas del libro
$n_paginas= encrypt(plaintext: htmlspecialchars(string: $_POST['n_paginas']));

```

Figura 3.3.0.4: Código de sanitización

3.4. Diseño inseguro

3.4.1. “hard-coded credentials”(credenciales en el código fuente)

El acceso no autorizado a las contraseñas almacenadas directamente en el código fuente del proyecto compromete la seguridad de estas ya que puede permitir a un atacante acceder a sistemas y datos privados. Antes de solucionarlo teníamos escrito todas las contraseñas en cada documento del proyecto, para solucionarlo lo que hicimos fue configurarlas como variables de entorno que se pueden cargar en el entorno de ejecución del sistema, sin necesidad de estar incluidas en el código fuente.

```

app > setup_sql.php
1  <?php
2
3  // conexión a la base de datos
4  //guarda el nombre del servidor a conectar
5  $servername = getenv('MYSQL_SERVER');
6  //guarda el nombre del usuario necesario para acceder al servidor
7  $username = getenv('MYSQL_USER');
8  //guarda la contraseña del usuario en una variable
9  $password = getenv('MYSQL_PASSWORD');
10 //guarda el nombre del de la base de datos a la que quiere acceder
11 $dbname = getenv('MYSQL_DATABASE');
12
13 //se realiza la conexión en el servidor con el usuario introducido en la base de datos introducida (db, database)
14 $conn = new mysqli($servername, $username, $password, $dbname);
15
16 // comprobar conexión
17
18 // si la variable que guarda la conexión es un error
19 if ($conn->connect_error) {
20     //detiene el proceso(die) e indica por pantalla la causa del fallo en la conexión
21     die("Connection failed: " . htmlspecialchars($conn->connect_error));
22 }

```

Figura 3.4.1.1: Variables de entorno

Asimismo elegimos una contraseña al azar para aumentar la seguridad de esta. De esta forma, las contraseñas estarán protegidas y aisladas del código fuente.

```

MYSQL_USER: admin
MYSQL_PASSWORD: WqoL30TBXD2ioZ15

```

Figura 3.4.1.2: Contraseña

3.4.2. Protección insuficiente de las cookies

Implementamos una configuración de cookies de sesión segura que habilita las opciones SameSite=Strict y HttpOnly=true. Esto aumenta la seguridad de las cookies de sesión y garantiza que no se envíen solicitudes entre sitios. Esto ayuda a prevenir ataques como la falsificación de solicitudes entre sitios (CSRF) y el clickjacking. Además, HttpOnly=true garantiza que no se puede acceder a las cookies a través de JavaScript. Esto protege la información sensible, como los tokens de sesión, de posibles ataques de robo por parte de scripts maliciosos

3.4.3. Reutilización de código seguro

La reutilización de código seguro y confiable es importante para la estandarización del código, evitando así posibles fallos al copiar múltiples veces un bloque de código. También ayuda a mejorar la sostenibilidad y el mantenimiento del código.

En nuestro caso, lo hemos aplicado en la reutilización de los archivos `comprobacionDeDatos.js` y `comprobarDatosLibro.js` que contienen las funciones de validación tanto para los usuarios como para los libros.

También se reutilizan las funciones de encriptado y desencriptado de los datos contenidas en el archivo `mysql_secret.php`.

También nos dimos cuenta que en todas las páginas se duplica el bloque de código encargado de establecer la conexión con la base de datos, por lo que se extrajo al archivo `setup_sql.php`. Pasaba lo mismo con el código encargado de establecer la sesión del usuario, por lo que se extrajo al archivo `setup_session.php`.

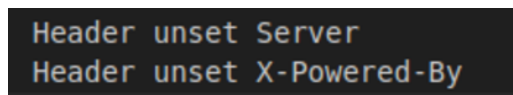
3.4.4. Comentarios difamatorios en los JavaScript

Uno de los fallos encontrados en zap es la difusión de información en comentarios en los documentos JavaScript. A pesar de mantenerlo oculto para todos los usuarios que intenten acceder desde la URL, se ha realizado una revisión del contenido de los comentarios. Tras analizar los comentarios hemos llegado a la conclusión de que estos no exponen ninguna información confidencial. Por lo tanto, los mantendremos sin ninguna variación.

3.5. Configuración de seguridad insuficiente

3.5.1. Divulgación de información sensible sobre el servidor web

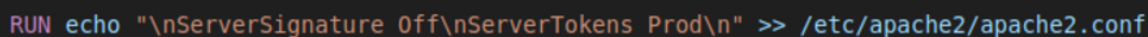
El encabezado `Server` en las respuestas HTTP revelaba detalles sobre el servidor (por ejemplo, si se estaba utilizando Apache o Nginx y la versión exacta), lo que proporcionaba a los atacantes información valiosa para buscar exploits específicos dirigidos a esta tecnología. Para solucionar este problema, se eliminaron el encabezado `Server` y el encabezado `X-Powered-By`, lo que evita que se revele información sobre el servidor y su software, lo que reduce la exposición de datos confidenciales.



```
Header unset Server
Header unset X-Powered-By
```

Figura 3.5.1.1: Encabezados

Además, la opción `ServerSignature` se estableció en `Off` y las opciones `ServerTokens` en `Prod` para minimizar la fuga de información del servidor web Apache.



```
RUN echo "\nServerSignature Off\nServerTokens Prod\n" >> /etc/apache2/apache2.conf
```

Figura 3.5.1.2: `ServerSignature` en `Off`

3.5.2. Encabezado `X-Content-Type`

En ZAP se nos mostraba el error de falta de encabezado `X-Content-Type`. Sin el encabezado `X-Content-Type-Options`, los navegadores pueden intentar adivinar el tipo de contenido de un archivo (por ejemplo, si un archivo JavaScript se trata como texto sin formato) si el tipo no se especifica explícitamente. Esto podría generar problemas de seguridad, como la ejecución de contenido malicioso o la fuga de información confidencial. Esto es especialmente riesgoso en versiones anteriores de navegadores como Chrome, que pueden ser más susceptibles a estos problemas.

Lo solucionamos implementando el encabezado `X-Content-Type-Options`: `nosniff`, este encabezado de seguridad se configuró en el servidor para evitar que los navegadores intenten "adivinar" el tipo de contenido de los archivos. Esto obliga a los navegadores a respetar el tipo MIME especificado en las respuestas HTTP y evita que se ejecuten archivos con tipos incorrectos. Al agregar este encabezado, los riesgos específicos en versiones

anteriores de Chrome que no aplicaban correctamente las políticas de tipo de contenido se solucionó, lo que mejoró la seguridad en esos navegadores.

```
Header always set X-Content-Type-Options
"nosniff"
```

Figura 3.5.2.1: Encabezado X-Content-Type-Options

3.5.3. Protección insuficiente contra framing

El encabezado X-Frame-Options es necesario para proteger las aplicaciones web de ataques de clickjacking al controlar si una página web se puede incrustar en un iframe en otros sitios, evitando así superposiciones maliciosas que engañen a los usuarios para que realicen acciones no deseadas.

Hasta ahora, nuestra página no tenía un encabezado de este tipo, que ahora configuramos como "SAMEORIGIN" a través de la configuración de Apache. Esto evita que otros sitios web framing nuestro sitio.

```
Header always set X-Frame-Options
"SAMEORIGIN"
```

Figura 3.5.3.1: Encabezado X-Frame-Options

3.5.4. Gestión de errores para no desvelar información

Es muy importante gestionar bien los errores que puedan surgir durante la ejecución de un sistema web, porque si no puede llegar a desvelarse información sensible que puede ser utilizada para ataques. Por eso, hay que limitar la información que se muestra por pantalla cuando surge un error.

En la primera entrega los mensajes que se mostraban cuando surgía un error eran demasiado informativos y, aunque fuese útil para nosotros ver qué era lo que había fallado con un simple window.alert, hay formas más seguras de gestionar esos errores.

Para esta entrega, hemos incluido en el código la estructura de control de errores try-catch, para que cuando una instrucción sql falle y salte una excepción no muestre el error por pantalla y maneje la excepción en el catch. Además, tanto ese tipo de errores como otros que pueden producirse durante la ejecución del sistema web son guardados en un archivo de errores (errores.log) al que no se puede acceder directamente desde la web. Así, los errores quedan registrados para que los administradores puedan gestionar los errores.

```
//rutas de los archivos de log
$error_file = '/var/www/logs/errores.log';
```

Figura 3.5.4.1: Ruta de archivo de errores

```
// Preparar la consulta SQL (utilizando prepared statements para prevenir inyecciones SQL)
$sql = "UPDATE usuarios SET nombre = ?, apellido = ?, telefono = ?, nacimiento = ?, email = ?, usuario = ? WHERE idUsuario = ?";

$stmt = $conn->prepare($sql);
$stmt->bind_param('ssssss', $nuevo_nombre, $nuevo_apellido, $nuevo_telefono, $nueva_fecha, $nuevo_email, $nuevo_usuario, $SESSION['user_id']);

// Ejecutar la consulta
try {
    $stmt->execute();
    unset($SESSION['csrf_token']);
    echo "<script>
        window.alert('Cambios guardados correctamente.');

```

Figura 3.5.4.2: Ejemplo de try catch

3.5.5. Cabecera CSP

La cabecera CSP(Content Security Policy) es una capa adicional de seguridad que ayuda a detectar y mitigar ciertos tipos de ataques, incluidos Cross Site Scripting (XSS) y ataques de inyección de datos. Estos ataques se utilizan para todo, desde el robo de datos hasta la desfiguración del sitio o la distribución de malware.

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'none';
              script-src 'self' 'nonce-abc123' ;
              style-src 'self' 'nonce-abc123' ;
              img-src 'self' http://localhost:81/image/background.jpg ;
              form-action 'self';">
```

Figura 3.5.5.1: Cabecera CSP

3.5.6. Pasar información por la URL(dirección web)

Desde la página de ítems.php, si se quería mirar o editar un libro en concreto se pasaba información sensible del libro por medio de la URL. Este método permitía a los usuarios visualizar y obtener información sensible del sistema sin necesidad de acceder a la base de datos.

Para solucionar el problema de la difamación de información sensible, se han implementado unos formularios que realizan las funciones de los anteriores botones. Los formularios, al pulsarse, se encargan de guardar la información sensible en la variable de sesión en vez de pasarla de forma visible por la URL. De esa manera, la información de la ID y el ISBN puede ser utilizada al igual que se ha hecho con la información en la URL, pero respetando la difusión de esta.

```
<td>
  <!--referencia as show_item.php cargado con el ISBN del libro-->
  <form method='POST' action='show_item.php'>
    <input type='hidden' name='csrf_token' value='" . htmlspecialchars(string: $_SESSION['csrf_token']) . "'>
    <input type='hidden' name='ISBN' value='" . decrypt(plaintext: htmlspecialchars(string: $row['ISBN'])) . "'>
    <button type='submit' class='button'>
      <img src='$nombimagen' class='imagen_catalogo'>
    </button>
  </form>
</td>
<!--informacion del titulo del libro-->
<td>" . decrypt(plaintext: htmlspecialchars(string: $row['titulo'])) . "</td>
<!--informacion del autor del libro-->
<td>" . decrypt(plaintext: htmlspecialchars(string: $row['autor'])) . "</td>
<td>
  <!--contenedor de los botones de modificacion y eliminacion con su respectivas imagenes-->
  <div class=button-container>
    <form method='POST' action='modify_item.php'>
      <input type='hidden' name='csrf_token' value='" . htmlspecialchars(string: $_SESSION['csrf_token']) . "'>
      <input type='hidden' name='idLibro' value='" . htmlspecialchars(string: $row['idLibro']) . "'>
      <button type='submit' class='button'>
        <img src='image/editar.png' class='imagen_funcionalidades'>
      </button>
    </form>
    <form method='POST' action='delete_item.php'>
      <input type='hidden' name='csrf_token' value='" . htmlspecialchars(string: $_SESSION['csrf_token']) . "'>
      <input type='hidden' name='ISBN' value='" . htmlspecialchars(string: $row['ISBN']) . "'>
      <button type='submit' class='button'>
        <img src='image/borrar.png' class='imagen_funcionalidades'>
      </button>
    </form>
  </td>
```

Figura 3.5.6.1: Formularios libros

3.5.7. Cabecera Cache-Control

Hemos añadido la cabecera Cache-Control que dictamina cómo se debe almacenar la página web en la caché del navegador. Hemos establecido en la cabecera el valor 'no-store' para que no se almacene nada en el navegador.

```
<meta http-equiv="Cache-Control" content="no-store, no-cache, must-revalidate">
```

Figura 3.5.7.1: Cabecera Cache-Control

3.6. Componentes vulnerables y obsoletos

3.6.1. Actualización de versión de PHP

El uso de versiones obsoletas de PHP (como la 7.2.2, que ya no recibe soporte) pueden exponer la página web a ataques y problemas de seguridad. Así que lo solucionamos actualizando la imagen de Docker para usar PHP 8.3 en lugar de la versión 7.2.2.

PHP 8.3 incluye mejoras de rendimiento y seguridad que ayudan a mitigar las vulnerabilidades conocidas de las versiones anteriores.

```
FROM php:8.3-apache-bookworm
```

Figura 3.6.1.1: Versión apache

3.6.2. Actualización de versión de phpmyadmin

Uno de los problemas encontrados con los componentes del sistema es que no se sabe exactamente qué versión tienen todos los componentes. phpmyadmin tiene la versión latest. Esto ocasiona un problema dentro de la seguridad porque no sabemos la versión a que ataques es vulnerable y, por lo tanto, no sabemos cómo proteger el sistema. Para solucionarlo, se ha actualizado la imagen de Docker para usar su versión más reciente sin que esta cambie de forma automatizada. image: phpmyadmin/phpmyadmin:5.2.1

3.7. Fallos de identificación y autenticación

3.7.1. Contraseñas débiles o por defecto

El uso de contraseñas débiles a la hora de registrarte en un sistema supone un problema que afecta tanto al usuario como al sistema. Al ser las contraseñas débiles, los atacantes acceden a las cuentas de otros usuarios con mayor facilidad utilizando ataques como ataques por fuerza bruta o ataque de diccionario.

Para mitigar el problema del acceso por estos ataques, se ha implementado dentro del JavaScript el método de comprobación de contraseña. Mediante ese método, el sistema determina si una contraseña es débil o no.

En caso de que la contraseña sea débil, el sistema no realizará el guardado de los datos y pedirá al usuario que introduzca una nueva contraseña que cumpla con las características deseadas(Al menos una letra mayúscula, una letra minúscula, un carácter especial, un número, más de 8 letras y no puede tener caracteres repetidos de forma consecutiva).

```

//Comprobacion de la contraseña
function comprobarContraseña(form) {
  //determinar qué formulario se está utilizando
  var contraseña;
  if (form.name === "register_form") {
    //si es el formulario de registro, usamos 'contraseña'
    contraseña = form.contrasena.value.trim();
  } else if (form.name === "user_modify_password") {
    //si es el formulario de modificación de usuario, usamos 'nuevacontrasena1'
    contraseña = form.nuevacontrasena1.value.trim();
  }

  if (contraseña.indexOf(' ') !== -1) {
    window.alert ("La contraseña no permite espacios en blanco");
    return false;
  }

  else if (contraseña.length < 8) {
    window.alert ("La contraseña debe tener al menos 8 caracteres");
    return false;
  }

  if (!/[A-Z]/.test(contraseña)) {
    window.alert ("La contraseña debe contener al menos una letra mayúscula");
    return false;
  }

  if (!/[a-z]/.test(contraseña)) {
    window.alert ("La contraseña debe contener al menos una letra minúscula");
    return false;
  }

  if (!/[0-9]/.test(contraseña)) {
    window.alert ("La contraseña debe contener al menos un número");
    return false;
  }

  if (!/[!@#$%^&*()_.,?":{}|<>]/.test(contraseña)) {
    window.alert ("La contraseña debe contener al menos un carácter especial");
    return false;
  }

  if (/(\.)\1{2}/.test(contraseña)) {
    window.alert ("La contraseña no debe tener caracteres repetidos consecutivos");
    return false;
  }

  return true;
}

```

Figura 3.7.1.1: Validación de contraseñas

Además de eso, íbamos a comprobar que la contraseña no fuese una de las 10000 contraseñas más comunes, pero ya que la mayoría de las contraseñas más usadas no cumplen las condiciones que hemos puesto en nuestro código, hemos decidido que no hace falta.

3.7.2. Invalidación de sesiones después de logout o inactividad

El mantener la sesión iniciada durante un tiempo prolongado facilita la obtención de la sesión y con ello realizar ataques en la cuenta iniciada. Para evitar este tipo de problemas de seguridad es necesario invalidar la sesión o bien por la desconexión del usuario (logout) o por tiempo de inactividad, si el usuario no cierra sesión.

Logout:

Este tipo de invalidación de sesión está implementado desde la anterior entrega. El programa ha sido implementado de tal manera que el usuario puede cerrar la sesión pulsando el botón ubicado en show_user. Este botón detecta que ha sido pulsado y comienza y elimina toda la información introducida en la variable de sesión.

```

//comprueba si se pulsa el botón de cerrar sesión
if (isset($_GET['action']) && $_GET['action'] == 'logout') {
  //borra las variables de sesion
  session_unset();
  //elimina la sesión
  session_destroy();

  echo "<script> window.location.href = 'index.php';</script>";
  exit();
}

```



Figura 3.7.2.1: Botón de cerrar sesión

Tiempo inactivo:

Al contrario del apartado anterior en este caso no se había hecho la implementación de la Invalidación de sesiones después de un tiempo inactivo. Para no comprometer la sesión del usuario se ha implementado debajo de `session_start()` un apartado que después de 20 minutos inactivo, la sesión del usuario se invalida automáticamente.

```
//se encarga de realizar un conteo del tiempo no activo, si pasan 20 minutos se cierra sesion
$tiempoExpiracion = 1200;//20 minutos

if (isset($_SESSION['last_activity'])) {
    if (time() - $_SESSION['last_activity'] > $tiempoExpiracion) {
        session_unset();
        session_destroy();
    }
}
$_SESSION['last_activity'] = time();
?>
```

Figura 3.7.2.2: Verificación de tiempo inactivo

3.7.3. Limitar intentos fallidos de login

Los ataques de fuerza bruta es un método que usan los atacantes para loguearse probando combinaciones de nombres de usuario y contraseñas. Por eso, es importante limitar los intentos de logins cuando se registran varios logins fallidos seguidos. El sistema web de la primera entrega permitía intentar loguearse tantas veces como se quisiera sin límites. Para arreglarlo, se ha establecido un límite de intentos de login y un tiempo de bloqueo. Cuando un usuario tenga 5 intentos fallidos de login, se le bloqueará el acceso a la página de login durante 5 minutos. Una vez transcurrido el tiempo de bloqueo, se reiniciarán los intentos fallidos y el usuario podrá intentar loguearse otra vez.

```
//limite de intentos fallidos
$intentos_maximos = 5; //5 intentos para logearse
$bloqueo_duracion = 300;//300; //5 minutos de bloqueo si el usuario pone 5 veces datos incorrectos al logearse

//inicializar el contador de intentos fallidos
if (!isset($_SESSION['intentos_fallidos'])) {
    $_SESSION['intentos_fallidos'] = 0;
}
if (!isset($_SESSION['tiempo_bloqueo'])) {
    $_SESSION['tiempo_bloqueo'] = 0;
}

//comprobamos si el usuario está bloqueado
if ($_SESSION['intentos_fallidos'] >= $intentos_maximos) {
    $tiempo_transcurrido = time() - $_SESSION['tiempo_bloqueo'];
    if ($tiempo_transcurrido < $bloqueo_duracion) {
        $espera = $bloqueo_duracion - $tiempo_transcurrido;
        $espera_minutos = ceil($espera / 60);
        if ($espera_minutos==1){echo "<script> window.alert('Demasiados intentos fallidos. Intenta de nuevo en {$espera_minutos} minuto.');</script>";}
        else{echo "<script> window.alert('Demasiados intentos fallidos. Intenta de nuevo en {$espera_minutos} minutos.');</script>";}
        echo "<script> window.location.href = 'index.php';</script>";
    } else {
        //reiniciamos el contador cuando acabe el tiempo de bloqueo
        $_SESSION['intentos_fallidos'] = 0;
        $_SESSION['tiempo_bloqueo'] = 0;
    }
}
}
```

Figura 3.7.3.1: Comprobación de intentos y tiempo de bloqueo

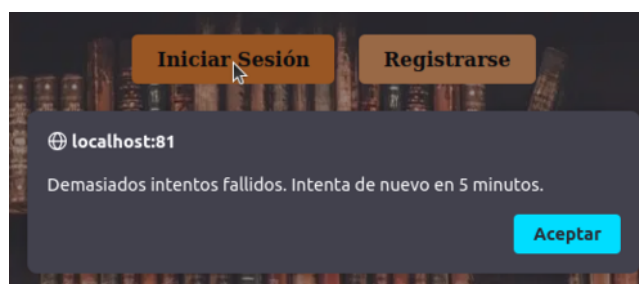


Figura 3.7.3.2: Mensaje mostrado tras 5 intentos fallidos

3.7.4. Logear intentos fallidos de login

Guardar los intentos fallidos de login que se producen puede ser muy útil para detectar posibles ataques de credential stuffing. El formato que se ha seguido para registrar los intentos de login es fecha, login fallido/exitoso, nombreUsuario, error, IP, user-agent y petición.

```

$ip_usuario = $_SERVER['REMOTE_ADDR'] ?? 'IP no disponible';
$user_agent = $_SERVER['HTTP_USER_AGENT'] ?? 'User-Agent no disponible';
//construir la petición del cliente
$metodo_http = $_SERVER['REQUEST_METHOD'] ?? 'UNKNOWN'; //metodo HTTP
$url = $_SERVER['REQUEST_URI'] ?? '/'; //URI solicitada
$version_http = $_SERVER['SERVER_PROTOCOL'] ?? 'HTTP/1.0'; //versión del protocolo
$peticion = "$metodo_http $url $version_http"; //construcción completa
//registrar intento fallido
file_put_contents(filename: $log_file, data: date(format: 'Y-m-d H:i:s') . " - Login fallido. Usuario: " .
  htmlspecialchars(string: $_POST['nombreUsuario']) . " Error: El usuario o la contraseña no coinciden. IP: " .
  htmlspecialchars(string: $ip_usuario) . ". User-Agent: " . htmlspecialchars(string: $user_agent) . ". Petición: " .
  htmlspecialchars(string: $peticion) . "\n", flags: FILE_APPEND);

```

Figura 3.7.4.1: Datos guardados en intentos_login.log

3.7.5. Uso de ID aleatorio para las sesiones

El uso continuo del número de la identidad asociado directamente al usuario supone un riesgo en la seguridad. Al hacer un uso de este identificador para verificar el inicio de sesión, es posible que un atacante obtenga este dato. En cambio, el utilizar una ID aleatoria para cada sesión, dificulta la obtención del identificador, disminuyendo el riesgo de ataques como el secuestro de sesión.

```

$_SESSION['randomID'] = bin2hex(string: random_bytes(length: 32));

```

Figura 3.7.5.1: ID aleatoria

3.7.6. Protección contra ataques Cross-Site Request Forgery (CSRF)

Nuestra sistema web era vulnerable a los ataques cross-site request forgery, también conocidos como ataques CSRF. En estos ataques, los ciberdelincuentes aprovechan la confianza del navegador y el servidor con la sesión activa del usuario. Aprovechando esa confianza, los atacantes hacen uso de páginas o solicitudes maliciosas para actuar sin consentimiento en nombre del usuario.

Para solucionarlo, se ha hecho uso de los tokens-csrf. Estos tokens son un número aleatorio único para cada sesión de usuario. En nuestro caso, el número aleatorio será un número aleatorio de 32 bytes. El número aleatorio es asignado de manera oculta al respectivo formulario de la página y se guarda su valor en la variable de sesión del usuario. De esa forma, se puede verificar la validez del formulario antes de procesar la solicitud.

```

if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(string: random_bytes(length: 32)); // Genera un token aleatorio seguro
}

```

Figura 3.7.6.1: Asignación del token en PHP

```

<body>
<form name="register_form" method="post" id="register_form">
  <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars(string: $_SESSION['csrf_token']); ?>">

```

Figura 3.7.6.2: Asignación del token en el formulario

```
// comprobar si se ha enviado el formulario
if (isset($_POST['register_submit'])) {
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
        $ip_usuario = $_SERVER['REMOTE_ADDR'] ?? 'IP no disponible';
        $user_agent = $_SERVER['HTTP_USER_AGENT'] ?? 'User-Agent no disponible';
        $metodo_http = $_SERVER['REQUEST_METHOD'] ?? 'UNKNOWN'; //metodo HTTP
        $uri = $_SERVER['REQUEST_URI'] ?? '/'; //URI solicitada
        $version_http = $_SERVER['SERVER_PROTOCOL'] ?? 'HTTP/1.0'; //versión del protocolo
        $peticion = "$metodo_http $uri $version_http"; //construcción completa
        file_put_contents(filename: $error_file, data: date(format: 'Y-m-d H:i:s') . " - Error CSRF. Error con el token en register. Token:"
        . htmlspecialchars(string: $_POST['csrf_token']) . " IP: " . htmlspecialchars(string: $ip_usuario) . ". User-Agent: "
        . htmlspecialchars(string: $user_agent) . ". Petición: " . htmlspecialchars(string: $peticion) . "\n", flags: FILE_APPEND);
        echo "<script>
            window.alert('no ha sido posible registrarse, pruebalo mas tarde');
            window.location.href = 'items.php';
        </script>";
        exit();
    }
}
```

Figura 3.7.6.3: Comprobacion del token

Para aumentar la seguridad de los tokens se borran su valor una vez se procesa la solicitud

```
unset($_SESSION['csrf_token']);
```

Figura 3.7.6.4: Retiro del token

3.7.7. Uso de reCAPTCHA

Hoy en día, es muy común encontrarse con ataques realizados por bots, especialmente en los accesos y registro a una página web. Estos bots intentan acceder a servicios en línea utilizando la suplantación de identidad, adivinación de contraseñas o envío masivo de formularios. Por esta razón, es esencial detectarlo y limitarse el acceso.

Para detectar y limitar este tipo de ataques, utilizaremos la API reCAPTCHA, concretamente reCAPTCHA V2. La API proporciona un desafío visual con el que determinará si el usuario es un humano o un robot. Este desafío se traduce en una puntuación de confianza entre 0 y 1, donde 1 es una alta probabilidad de que el usuario sea un humano y 0 de que sea un robot.

En lo que respecta a nuestra web, hemos tenido que añadir. Por un lado, hemos añadido los elementos de html que permitirá al usuario interactuar con el reCAPTCHA, haciendo uso de un botón para indicar que no es un robot. Por otro lado, hemos metido los elementos de php que permite enviar la solicitud a <https://www.google.com/recaptcha/api/siteverify>, donde se verificara la solicitud.

```

<?php // Verificación de reCAPTCHA

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    if (!isset($_POST['g-recaptcha-response']) || empty($_POST['g-recaptcha-response'])) {
        echo "<script>alert('verifique que no es usted un robot');</script>";
        echo "<script>window.location.href = '" . htmlspecialchars(string: $_SERVER['PHP_SELF']) . "';</script>";
        exit();
    }

    $captcha = $_POST['g-recaptcha-response'];

    if (!$captcha) {
        echo "<script>alert('Por favor completa el reCAPTCHA.');

```

Figura 3.7.7.1: Php de recaptcha

```

<div class="g-recaptcha" data-sitekey="<?php echo getenv(name: 'RECAPTCHA_WEB_SIDE'); ?>"></div>

```

Figura 3.7.7.2: Botón de nosoy un robot

```

<script nonce="abc123" src="https://www.google.com/recaptcha/api.js" async defer></script>

```

Figura 3.7.7.3: Script de comprobación

Es importante mencionar los cambios que se han tenido que realizar en la cabecera CSP(Content Security Policy) para poder acceder a la página de reCAPTCHA.Como la página de reCAPTCHA no se encuentra en nuestro dominio, mantener la política de CSP como estaba hace que la API recaptcha no pueda ser utilizada en nuestra web.

```

<meta http-equiv="Content-Security-Policy"
content="default-src 'none';
script-src 'self' https://www.google.com https://www.gstatic.com 'nonce-abc123';
style-src 'self' https://fonts.googleapis.com 'nonce-abc123';
img-src 'self' https://www.gstatic.com http://localhost:81/image/background.jpg ;
frame-src https://www.google.com;
form-action 'self';">

```

Figura 3.7.7.4: Política CSP

Al incluir páginas no pertenecientes a nuestro dominio, el CSP no es tan restrictivo. Es por eso que es más susceptible a ataques XSS(Cross-Site Scripting), que son ataques de inyección de código malicioso.

3.8. Fallos en la integridad de datos y software

3.8.1. Librerías no confiables

El uso de librerías de fuentes no confiables puede producir graves brechas de seguridad ya que dichas librerías pueden contener exploits o puertas traseras que comprometen la integridad del sistema. En nuestro caso no tenemos este problema ya que no usamos librerías externas.

3.8.2. Copias de Seguridad

Tener una buena gestión de las copias de seguridad es clave para garantizar la integridad de la información. Es por eso que nuestro grupo realiza copias de seguridad del proyecto en 2 soportes diferentes. El primero es la nube de github, donde el sistema permite actuar como herramienta que efectúa copias incrementales. Por otra parte, guardamos los cambios en un pendrive donde debido a su limitada capacidad se realizan copias integrales que ahorra espacio. Estas copias se realizan diariamente, por su gran cantidad de cambios a lo largo del día.

De esta manera, se cumple la regla de backups 3-2-1 donde hay 3 copias del proyecto(2 copias y 1 el proyecto) que se almacenan en 2 soportes diferentes, pendrive(físico) y github (nube) y 1 de la copias de seguridad se encuentra en otro lugar físico, muy probablemente en Madrid (github no da ese tipo de información).

3.9. Fallos en la monitorización de la seguridad

Dado que la seguridad es un proceso y no un estado es imperativo que se revise continuamente la seguridad del sistema. Este sistema de revisión y mejora continua se puede hacer a través de auditorías de seguridad. Aún así, la monitorización de la web es adecuada porque se registran todos los login junto a información que es útil para identificar qué o quién está intentando loguearse y en caso de tener 5 intentos fallidos la web es capaz de actuar bloqueando el acceso para loguearse.

4. Segunda auditoría

Después de estudiar y revisar los problemas vistos en la primera auditoría hemos realizado una segunda auditoría para comprobar que hemos resuelto las vulnerabilidades encontradas y que no han surgido nuevas.

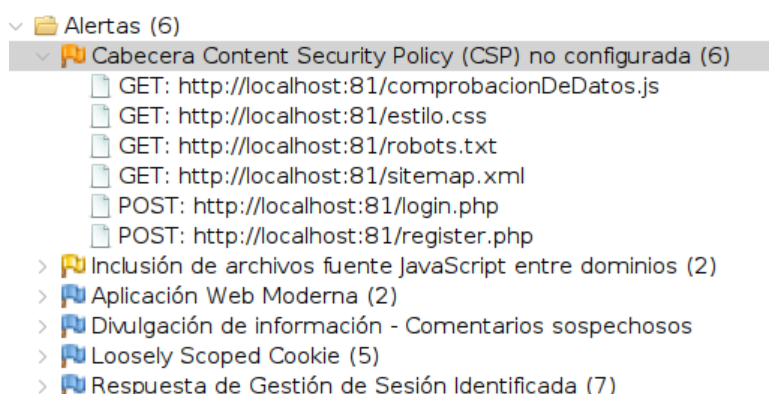


Figura 4.0.0.1: Resultado segunda auditoría

Podemos ver que se han resuelto todos los fallos graves y la mayoría de los fallos son menores. Cabe aclarar que consideramos que el fallo de CSP es falso positivo. Después de estudiar y analizar el código fuente no hemos encontrado la causa de dichos fallos por lo que se consideran arreglados.

5. Conclusiones

Haciendo una comparación entre ambas auditorías podemos ver una clara mejoría en la seguridad del sistema. Tanto con la herramienta ZAP como con pruebas realizadas por nosotros mismos hemos comprobado que los ataques que antes eran efectivos ya no lo son.

Antes cualquier usuario podía acceder a cualquier parte del sistema, incluyendo la base de datos, pudiendo modificar y recoger cualquier información. Tras las mejoras realizadas no solo hemos implantado un sistema de autorización y autenticación fuerte si no que también hemos asegurado la confidencialidad de toda la información gracias a la encriptación de todos los datos tanto en transmisión como en reposo.

Este proyecto nos ha hecho darnos cuenta de la importancia de tener una buena implementación en la seguridad web a lo largo de todo el desarrollo, y de la necesidad de realizar auditorías periódicamente, no sólo para resolver problemas existentes sino también para evitar problemas futuros.

6. Observaciones

En primer lugar, hemos observado que debido a la nueva ubicación de las claves de phpmyadmin, ubicadas en el entorno, hay un atraso en la carga de las contraseñas a la ora de utilizar la web. Para evitar este problema es recomendable mantenerlo unos segundos después del docker-compose up para que todas las funciones sean accesibles.

Relacionado con la ubicación de las contraseñas, cabe aclarar que por temas de solicitud del proyecto se han tenido que incluir todas las claves en repositorio público en git. Esta acción hace que las contraseñas establecidas queden comprometidas. Sin embargo, si el proyecto se hubiera realizado a modo de servidor, las claves se hubieran quedado ocultas para todo el mundo, haciéndolas seguras.

7. Bibliografía

- OWASP (2021). Top 10 vulnerabilidades de 2021 - <https://owasp.org/Top10/>
- GPT-4 (2024). Resolución de preguntas y aclaración de conceptos. - <https://chatgpt.com/>
- PHP. (2024). Manual de PHP. - <https://www.php.net/manual/es/index.php>
- reCAPTCHA
 - instancia - <https://www.google.com/recaptcha/about/>
 - información - <https://developers.google.com/recaptcha/docs/display?hl=es-419>